

[Abhay · 0:20]

SAY

Our project answers one business question: how much money should an investor put into a portfolio of three Indian stocks, given how much risk they can bear? All of the code is in one Python file, and our tests check that it matches our Excel workbook and our larger pipeline.

WHAT THE CODE IS DOING

1. python_project.py has 1,108 lines in 14 numbered sections, and 21 tests check it.
2. It reads frozen data, so the numbers on screen match these slides.
3. It imports nothing else from our repository, so all the logic is in this one file.

IF YOU BLANK

One file takes an investor's answers and gives back a rupee split and a measure of risk.



[Abhay · 0:40]

SAY

The program reads four kinds of data: prices and returns, the inputs from our Excel valuation workbook, a settings file, and text. It takes an investor's ten questionnaire answers and an amount, and gives back a rupee split. A balanced investor with one lakh holds ₹21,833 of ICICI, for example, with a one-year 95% Value at Risk of ₹5,582.

WHAT THE CODE IS DOING

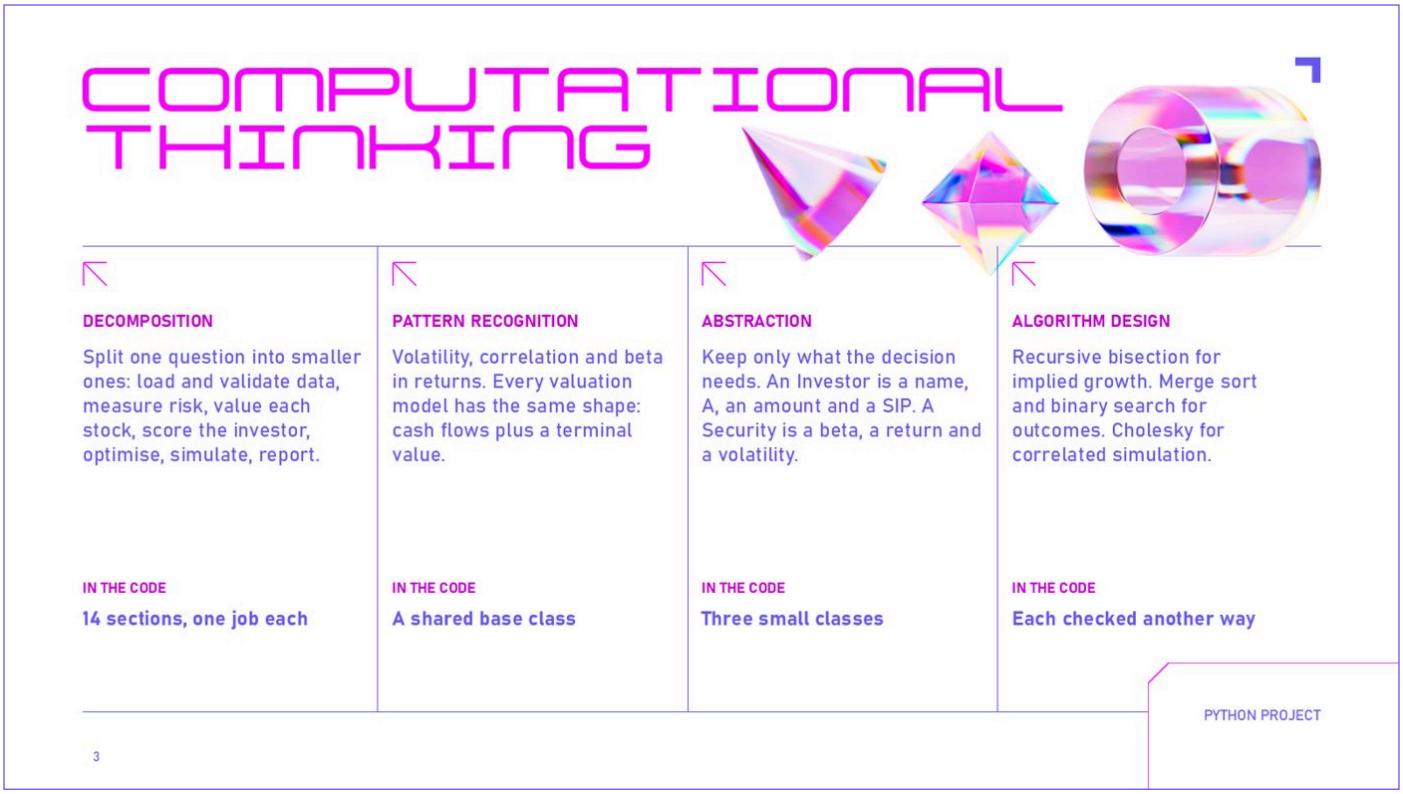
1. load_project reads the settings, returns and portfolio data.
2. risk_aversion_from turns the ten answers into a number, A. Low A means more risk is acceptable.
3. max_sharpe_weights finds the best mix of the three stocks.
4. Portfolio.complete decides how much goes into that mix and how much into T-bills.
5. run_simulations and risk_summary simulate 10,000 paths and measure the risk.

IF YOU BLANK

Answers become A, A becomes a share in the portfolio, and the share becomes rupees and a risk figure.

IF ASKED

VaR: in 95% of simulated years, a balanced investor loses no more than this amount.



[Abhay · 0:45]

SAY

We followed the four computational-thinking stages from our proposal. Decomposition: one large question became smaller ones, which is why the file has fourteen sections. Pattern recognition: every valuation model is cash flows plus a terminal value, and that repeated shape became a base class. Abstraction: an investor needs only a name, a risk-aversion number, an amount and a SIP. Algorithm design: each algorithm is checked against a second method.

WHAT THE CODE IS DOING

1. Sections run in order: settings, files, risk measures, questionnaire, classes, valuation, sorting, optimiser, simulation, rules, text, charts, tour, demo.

2. tour() at the end calls them all and prints each result next to the pipeline's, marked OK.

IF YOU BLANK

We broke the problem down, spotted what repeats, kept only what matters, and wrote algorithms we could check.

CONCEPT MAP



EVERY COURSE CONCEPT, MAPPED TO THE FILE. EACH ONE DOES REAL WORK IN THE PROJECT.

Concept	Section	What it does in the project
Variables, numbers, strings	1	Settings; Indian-style rupee formatting
Files and exceptions	2	JSON, YAML, CSV, Parquet, PDF; a clear DataError
Functions, Pandas	3	Returns, Sharpe, beta, Treynor, Jensen's alpha
Decisions, loops, dictionaries	4, 10	Questionnaire scoring; trading-rule signals
Classes	5	Security, Investor, Portfolio
Inheritance, recursion	6	ValuationModel and two subclasses; recursive PV, bisection
Sorting and searching	7	Merge sort, binary search, grid search
SciPy	8	Maximum Sharpe weights, efficient frontier
NumPy	9, 10	Cholesky Monte Carlo, block bootstrap, EWMA, SIP
Sets, Transformers, pypdf, yfinance	2, 11	Hedge words, FinBERT, PDF text, price download
Matplotlib, Seaborn	12	Frontier, fan chart, correlation heatmap
Streamlit	14	A small interactive demo in the same file

[Abhay · 0:40]

SAY

This is the map of the file, and it's the same list at the top of the code. Every topic in the course outline appears here, and each one does real work rather than being a separate exercise. Sections 5, 6 and 7 are the ones we'll show you in code, because they are the core of the course.

WHAT THE CODE IS DOING

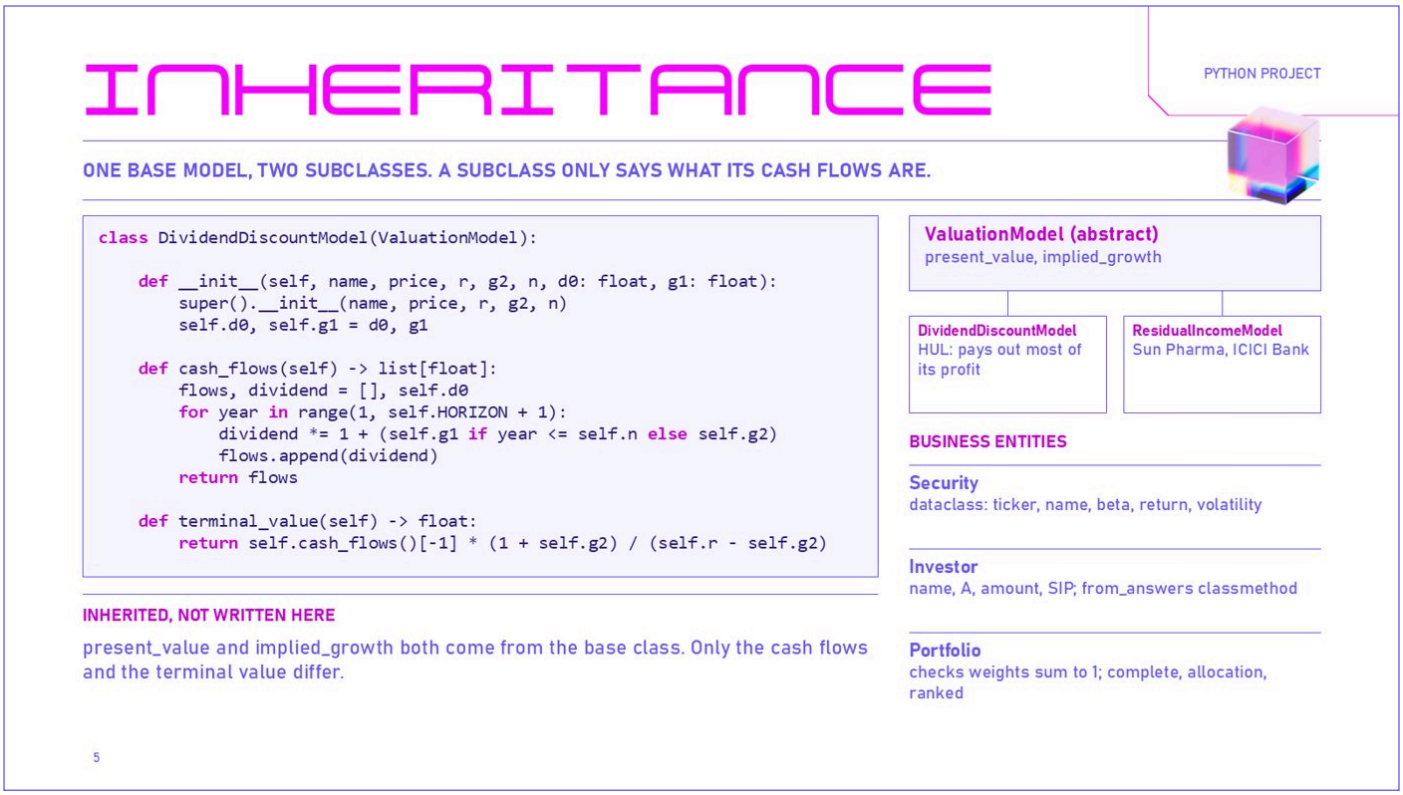
1. The docstring at the top of python_project.py holds this same list, so anyone reading the file can find a concept quickly.

2. Heavy libraries (Transformers, Seaborn, yfinance) are imported inside the function that needs them.

IF YOU BLANK

Every concept from the course is used here to do a real job in the project. Pick any row and we can open it.

WHERE: python_project.py, the map is at lines 19–31.



[Abhay · 1:00]

SAY

Inheritance is justified here because the models really do share logic. The base class, ValuationModel, is abstract: it discounts cash flows, solves for implied growth, and refuses bad inputs such as a discount rate below growth. Each subclass only says what its cash flows are. HUL uses this dividend model because it pays out most of its profit. Sun Pharma and ICICI use residual income.

WHAT THE CODE IS DOING

1. class DividendDiscountModel(ValuationModel) means: a child that gets all of its parent's methods.

2. __init__: super().__init__ runs the parent's checks (r must exceed g2), then the child stores today's dividend d0 and the growth rate g1.

3. cash_flows: start from d0, loop over years 1 to 10, grow by g1 in the high-growth years and by g2 after, and collect each dividend in a list.

4. terminal_value: the Gordon growth formula on the year-10 dividend, the value of everything after year 10.

5. intrinsic_value, in the parent, adds the present value of the flows and the terminal value. The child never rewrites that.

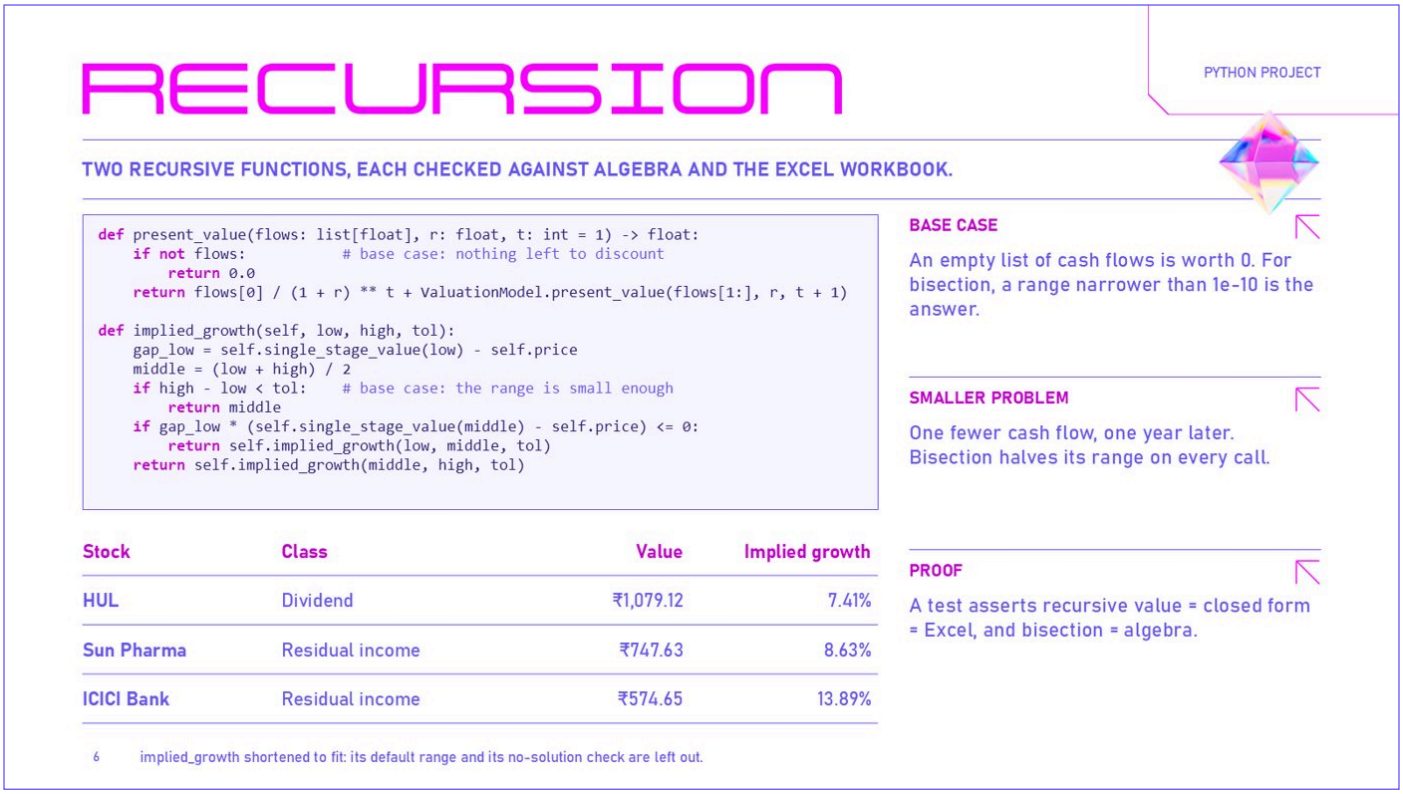
IF YOU BLANK

The parent does the maths. The child only says what the cash flows are.

IF ASKED

Abstract means the parent can't be created by itself, and neither can a child that leaves out a required method. A test checks this.

WHERE: python_project.py, ValuationModel line 388, DividendDiscountModel line 454, ResidualIncomeModel line 483.



[Abhay · 1:00]

SAY

Recursion appears twice. The present value of a list of cash flows is the first flow discounted, plus the present value of the rest. Then bisection: what growth rate makes the model equal the market price? We take the middle of a range, see which half contains the answer, and call the function again on that half. A loop would work too, so why trust it? Because the recursive value equals the closed-form formula and Excel, and bisection equals the algebra.

WHAT THE CODE IS DOING

1. present_value: if the list is empty, return 0 (the base case). Otherwise discount the first flow and call itself on the rest, one year later.

2. Example: flows 10 and 11 at r = 10%: 10/1.1 + 11/1.21 + 0 = 18.18.

3. implied_growth finds the growth rate where value equals price. gap_low is the gap at the bottom of the range.

4. If the gap changes sign between low and middle, the answer is in the lower half, so it calls itself on that half. Otherwise it uses the upper half.

5. When the range is narrower than tol (1e-10), it returns the middle. That took 34 calls on each of our stocks.

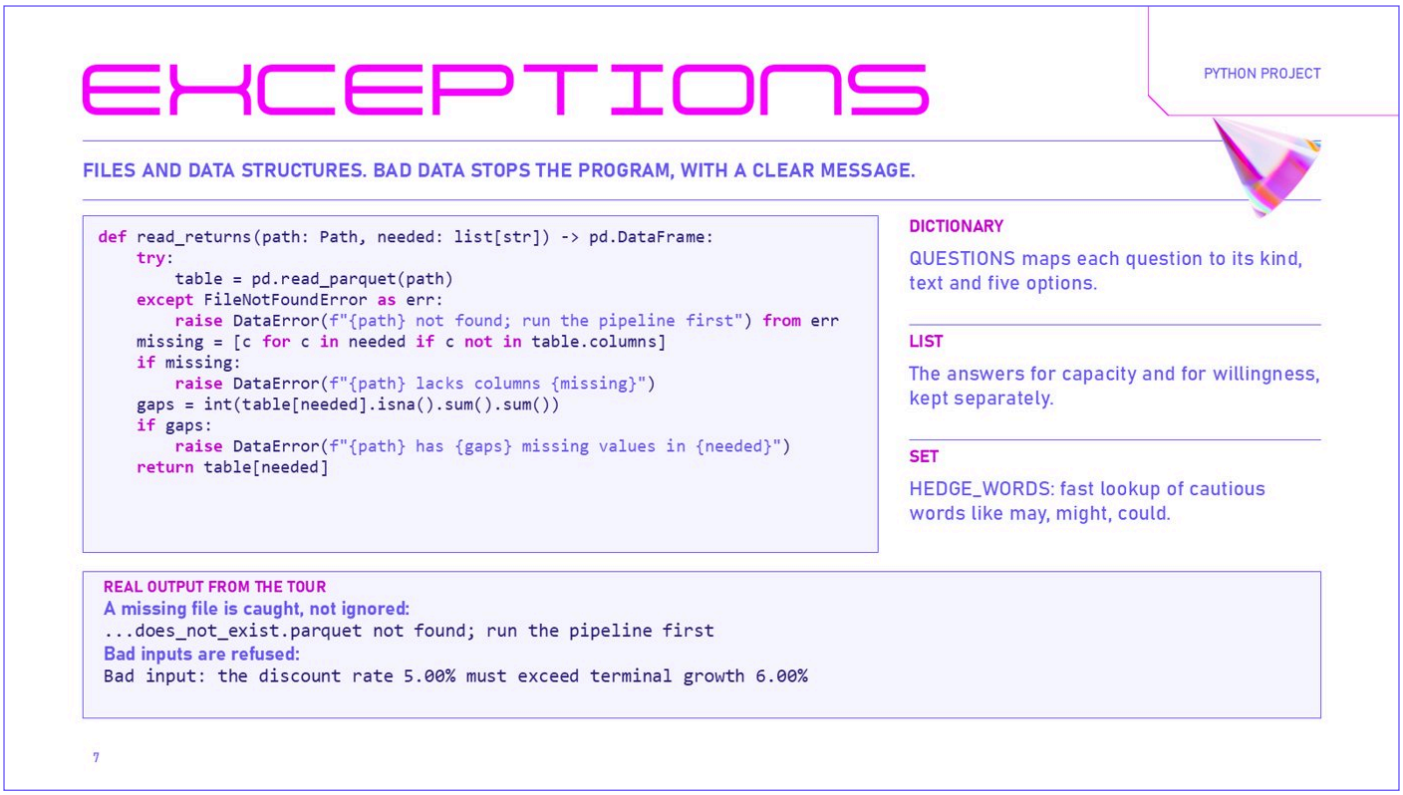
IF YOU BLANK

Each call does one small step and hands a smaller problem to itself, until nothing is left.

IF ASKED

Python's default limit is about 1,000 nested calls. These go 11 and 34 deep, so it is safe.

WHERE: python_project.py, present_value line 404, implied_growth line 416.



[Abhay · 0:40]

SAY

Financial data is messy, so we decided that bad data should stop the program rather than be filled with a guess. This function reads a returns file and raises our own DataError if the file is missing, a column is missing, or there are gaps. The data structures are chosen for a reason: a dictionary for the questionnaire, lists for the answers, and a set for hedge words because lookup in a set is fast. The messages at the bottom are real output.

WHAT THE CODE IS DOING

1. try / except: try to read the Parquet file. If it isn't there, raise our DataError saying which file and what to do.
2. from err keeps the original error attached, which helps debugging.
3. missing: a list comprehension collects any column we need that the file lacks.
4. gaps: counts empty cells in those columns. Any gap stops the program.
5. If all is well, it returns only the columns we asked for.

IF YOU BLANK

We check the file, the columns and the gaps. If anything is wrong we stop and say what, because a guess would silently change every number.

IF ASKED

The file reads JSON, YAML, CSV, Parquet, Excel and PDF, each with its own error handling.

WHERE: python_project.py, DataError line 76, read_returns line 101, QUESTIONS line 239, HEDGE_WORDS line 809.

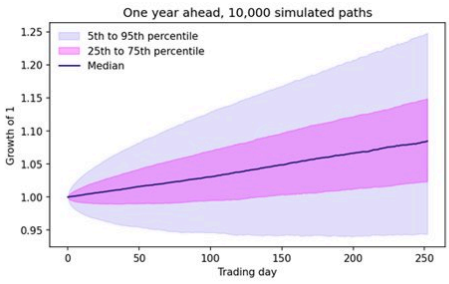
SIMULATION

PYTHON PROJECT

NUMPY: 10,000 SIMULATED YEARS, WITHOUT A PYTHON LOOP OVER THE PATHS.

```python
def simulate_normal(mu, cov, n_sims, horizon, rng):
    chol = np.linalg.cholesky(cov)
    return rng.standard_normal((n_sims, horizon, len(mu))) @ chol.T + mu

def wealth_paths(risky_returns, y, rf_period):
    growth = np.cumprod(1 + y * risky_returns + (1 - y) * rf_period, axis=1)
    return np.hstack([np.ones((risky_returns.shape[0], 1)), growth])
```



₹1,08,463
MEDIAN AFTER ONE YEAR, ₹1 LAKH, A = 4

₹5,582
95% VaR, NORMAL RETURNS

₹5,465
95% VaR, BLOCK BOOTSTRAP

CHOLESKY

Split the covariance matrix so that cov = L L^T. Independent draws multiplied by L have the correlations we measured.

Half of all paths end between ₹1,02,377 and ₹1,14,860.

8 Type hints trimmed to fit the slide.

[Sankarshana · 1:00]

SAY

We need thousands of possible futures in which the three stocks move together realistically. Independent random numbers wouldn't do that, so the Cholesky factor of the covariance matrix turns them into correlated returns. These are NumPy arrays of shape paths by days by stocks, so one line handles all 10,000 paths. We also resample real history in blocks, which keeps fat tails. On one lakh, the 95% Value at Risk is ₹5,582 with normal returns and ₹5,465 with the bootstrap.

WHAT THE CODE IS DOING

1. chol: the Cholesky factor of the covariance matrix.

2. rng.standard_normal gives independent random draws: one number per path, per day, per stock.

3. @ chol.T builds in the correlation, and + mu adds the average daily return.

4. wealth_paths: each day's growth is 1 + y x (portfolio return) + (1 - y) x (T-bill return), so y is the share in the stocks.

5. np.cumprod multiplies those along each path, day by day. hstack puts the starting value of 1 in front.

6. VaR is the amount invested minus the 5th percentile of the final values.

IF YOU BLANK

We simulate 10,000 correlated years at once with arrays, then read the risk off the worst outcomes.

IF ASKED

The seed is fixed at 42 in config.yaml, so results repeat exactly and match mc.json. It is a simulation under CAPM returns, not a forecast.

WHERE: python_project.py, simulate_normal line 658, block_bootstrap line 664, wealth_paths line 680, risk_summary line 686.

OPTIMISATION

PYTHON PROJECT

SCIPY FINDS THE MAXIMUM-SHARPE WEIGHTS. A 1% GRID SEARCH FINDS THE SAME PORTFOLIO.

```
def max_sharpe_weights(mu, cov, rf: float, bounds=WEIGHT_BOUNDS) -> np.ndarray:
    mu, cov = np.asarray(mu), np.asarray(cov)
    return _optimise(lambda w: -(w @ mu - rf) / np.sqrt(w @ cov @ w), len(mu), bounds)

def _optimise(objective, n: int, bounds, constraints=()) -> np.ndarray:
    all_constraints = [{"type": "eq", "fun": lambda w: w.sum() - 1.0}, *constraints]
    result = minimize(objective, np.full(n, 1.0 / n), method="SLSQP", bounds=[bounds] * n,
                      constraints=all_constraints, options={"ftol": 1e-12, "maxiter": 500})
```

SCIPY SLSQP

31 / 19 / 50

Sharpe 0.3436, within 10–60% limits

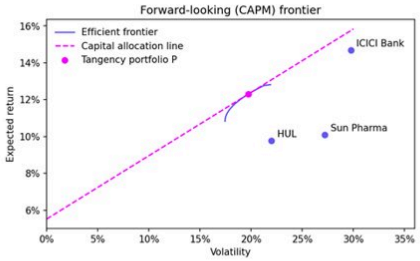
1% GRID SEARCH

31 / 19 / 50

Same portfolio, by trying every mix

THEN SIZE IT FOR EACH INVESTOR

y* = (E[rP] − rf) / (Aσ²): 87% at A = 2, 44% at A = 4, 29% at A = 6. The rest goes to T-bills.



9

[Sankarshana · 0:40]

SAY

SciPy's minimize function solves the constrained problem: maximise the Sharpe ratio, with weights that sum to one and stay between 10 and 60 percent. It gives 31, 19, 50. How do we know it worked? We wrote a plain grid search that tries every mix at 1% steps. It's slow, but it's easy to trust, and it finds the same portfolio. Then the formula on the slide sizes it for each investor using their risk aversion.

WHAT THE CODE IS DOING

1. The objective is the Sharpe ratio with a minus sign, because SciPy minimises. Minimising the negative maximises the Sharpe.

2. w @ mu is the portfolio's return. w @ cov @ w is its variance.

3. The constraint says the weights must sum to 1. bounds keeps each stock between 10% and 60%.

4. It starts from equal weights, and SLSQP is the method that handles limits and constraints. If it fails, we raise an error.

5. grid_search_max_sharpe loops over the weights of two stocks, gives the third what's left, and keeps the best Sharpe.

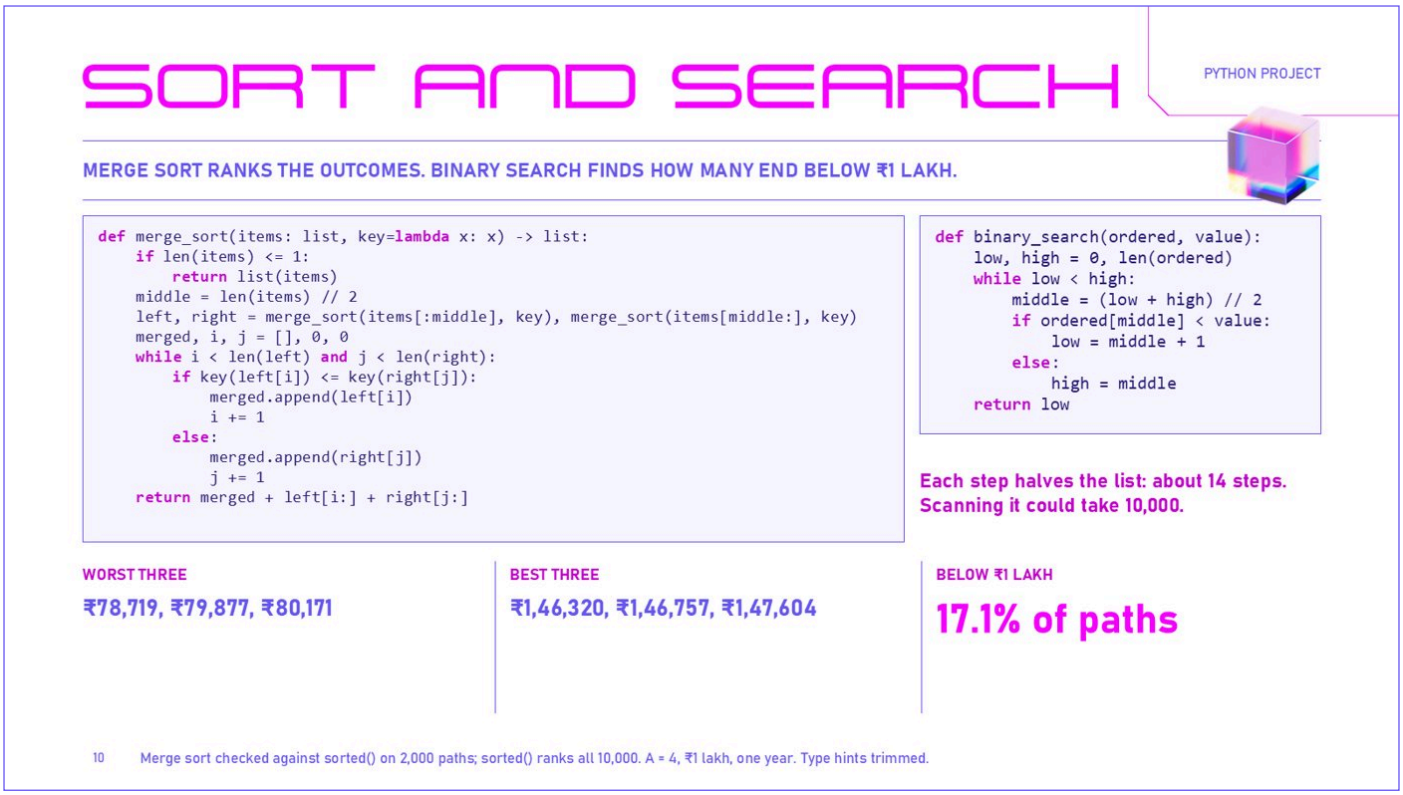
IF YOU BLANK

The optimiser searches for the steepest return per unit of risk, and a brute-force search confirms the answer.

IF ASKED

@ is matrix multiplication. Sharpe is excess return per unit of risk.

WHERE: python_project.py, max_sharpe_weights line 639, _optimise line 628, grid_search_max_sharpe line 608.



[Sankarshana · 0:40]

SAY

A risk manager cares about the tails, so we sort the simulated outcomes. Merge sort is recursive: sort each half, then merge them. We wrote it ourselves and check it against Python's sorted on 2,000 paths, then use Python's sorted for all 10,000, since a pure-Python recursive sort is slower. On the sorted list, binary search halves the range each step, so it finds how many paths end below one lakh in about fourteen steps.

WHAT THE CODE IS DOING

1. merge_sort: a list of 0 or 1 items is already sorted (the base case).

2. Otherwise split at the middle and sort each half by calling itself.

3. The while loop merges: compare the front of each half, take the smaller, move along. Whatever is left is added at the end.

4. binary_search: keep a range low to high. Look at the middle. If it's below the value, the answer is to the right; otherwise it's to the left.

5. It ends with low = how many items are below the value.

IF YOU BLANK

Sort by splitting and merging. Then search a sorted list by halving it each time.

IF ASKED

Merge sort is O(n log n) and binary search is O(log n). Binary search needs a sorted list.

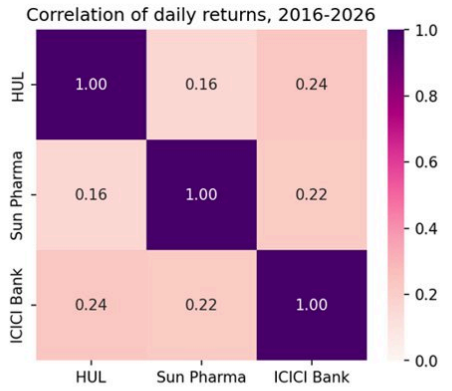
WHERE: python_project.py, merge_sort line 571, binary_search line 588, best_and_worst line 381.

LIBRARIES

PYTHON PROJECT

EACH LIBRARY HAS ONE JOB. HEAVY ONES LOAD ONLY INSIDE THE FUNCTION THAT NEEDS THEM.

Library	Job
Pandas	Load, align and summarise returns; metrics tables
Matplotlib	Efficient frontier and Monte Carlo fan chart
Seaborn	Correlation heatmap
yfinance	Fresh prices (--fetch)
Transformers (FinBERT)	Financial tone of a sentence (--finbert)
pypdf	Text from annual reports and transcripts (--pdf)
openpyxl	Excel valuation inputs, cell by cell
Streamlit	Interactive demo on one screen
requests	Optional live answers through OpenRouter



11

[Sankarshana · 0:30]

SAY

Each library has one clear job. Pandas handles the tables, Matplotlib and Seaborn draw the charts, like this heatmap, yfinance downloads prices, FinBERT reads tone, pypdf reads reports and Streamlit gives us the demo. One design choice: the heavy libraries are imported inside the function that needs them, so the file still runs on a machine that doesn't have them all.

WHAT THE CODE IS DOING

1. NumPy and Pandas are imported at the top because everything uses them.

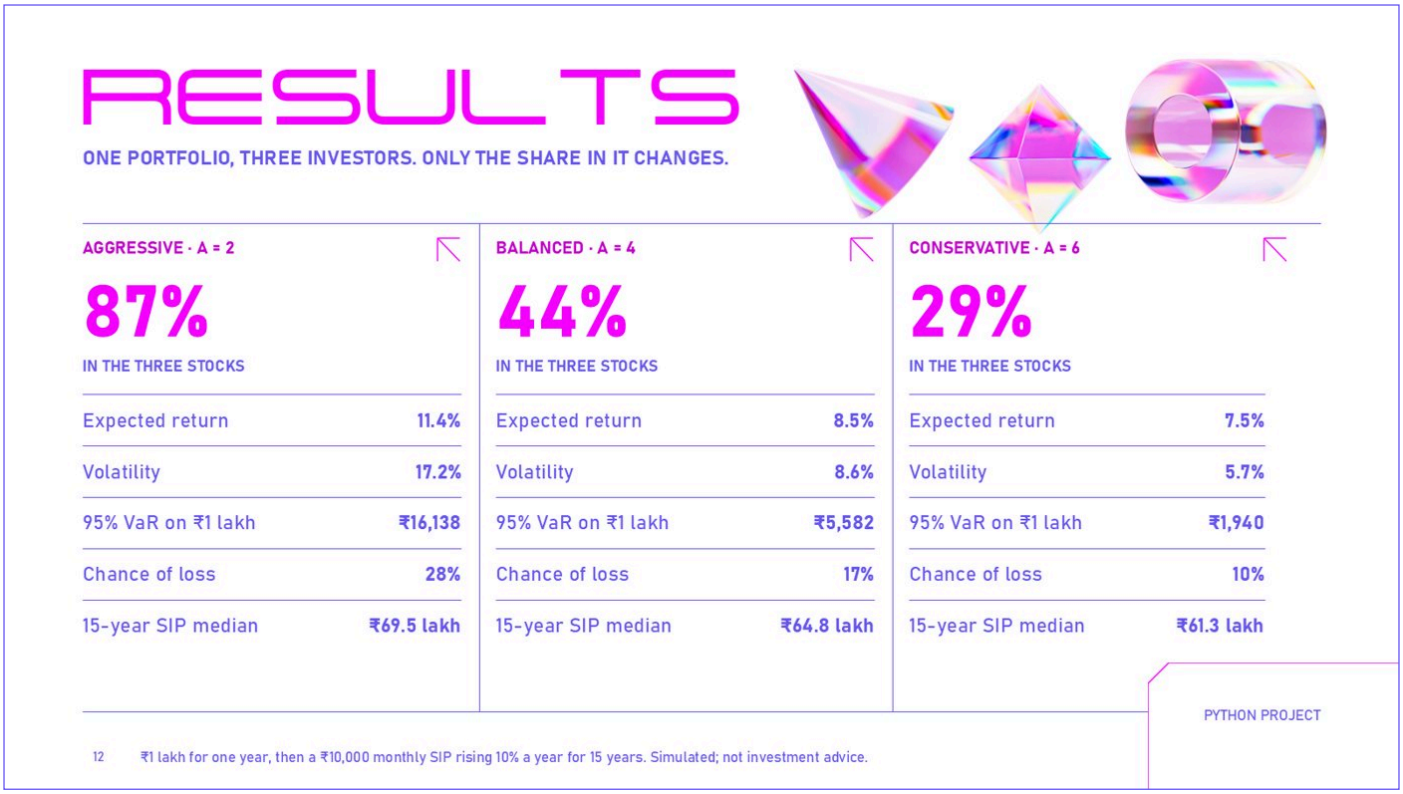
2. save_charts imports Matplotlib and Seaborn, and finbert_tone imports Transformers, only when called.

3. The heatmap shows correlations of 0.16 to 0.24, so the three stocks diversify each other.

IF YOU BLANK

Each library does one job, and optional ones load only when needed.

WHERE: python_project.py, download_prices line 143, finbert_tone line 825, save_charts line 858.



[Sankarshana · 0:40]

SAY

Here is what the program tells three different investors. They all hold the same mix of stocks, 31, 19, 50, because that is the best risky portfolio for everyone. What changes is how much of it they hold: 87%, 44% or 29%. This is the separation property from finance, and it's also a nice software result: one Portfolio object, three Investor objects.

WHAT THE CODE IS DOING

1. Portfolio.complete computes y* = (E[rP] - rf) / (A x variance), then limits it to between 0 and 1.
2. It then works out the investor's return and volatility: rf + y x (E[rP] - rf), and y x volatility.
3. Portfolio.allocation multiplies y by each weight and the amount to get rupees. The rest goes to T-bills.
4. Investor.from_answers builds the investor from the questionnaire, using a class method.

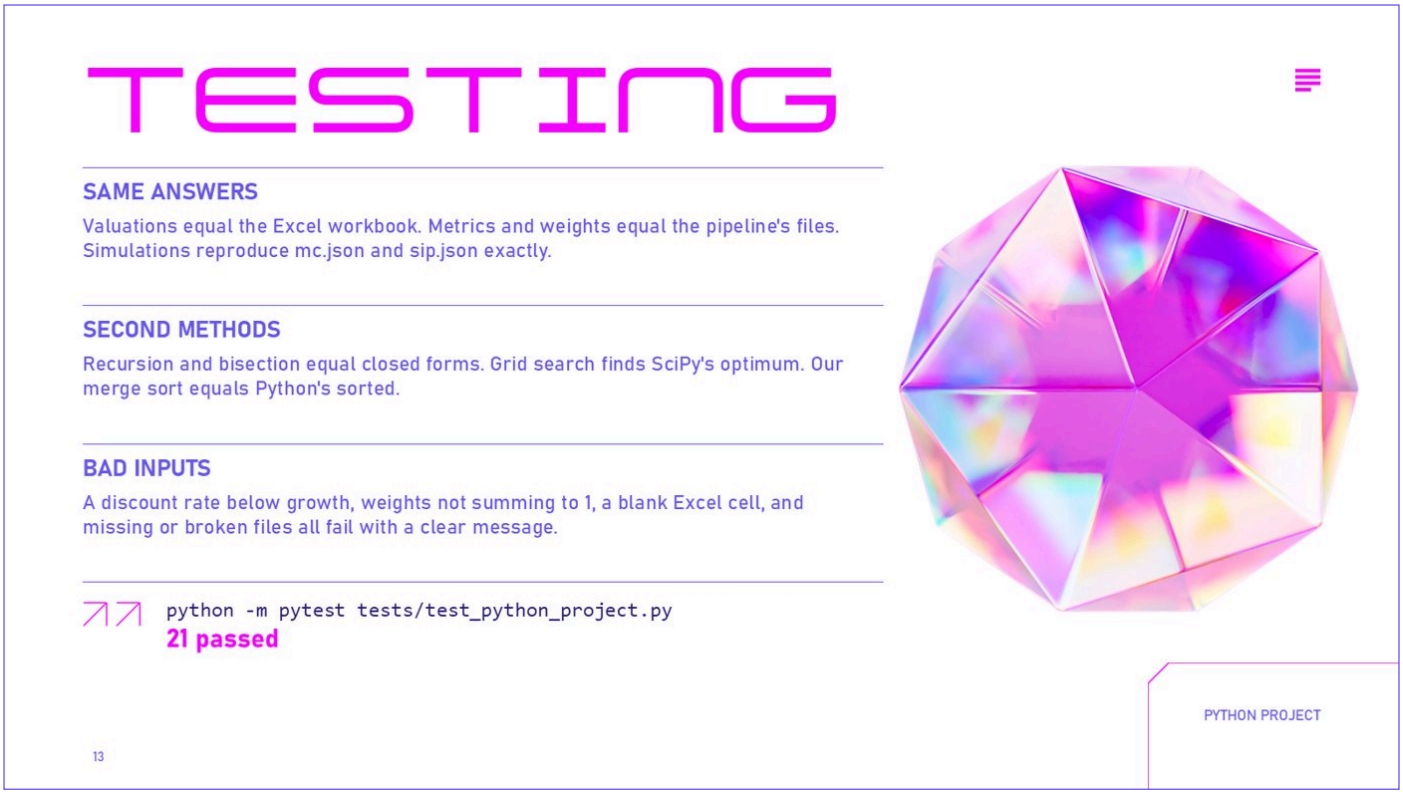
IF YOU BLANK

Everyone holds the same stocks. Risk aversion only decides how much of their money goes in.

IF ASKED

If A is 0 or below, risk doesn't reduce utility, so y* has no finite value and the investor holds the maximum allowed.

WHERE: python_project.py, Investor line 312, Portfolio line 329, complete line 354, allocation line 369.



[Sankarshana · 0:30]

SAY

We have 21 automated tests in three groups. First, the file must give the same answers as our Excel workbook and our main pipeline, down to the rupee. Second, each algorithm is checked by a different method. Third, bad inputs must fail with a clear message. We'll run them live in the demo.

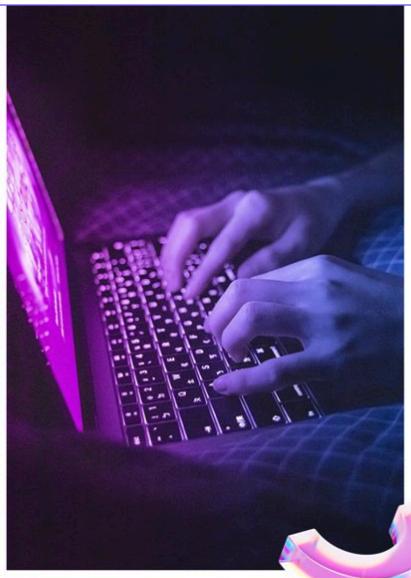
WHAT THE CODE IS DOING

1. pytest finds every function named test_ and runs it. assert stops the test if a result is wrong.
2. The simulation tests use the same seed and order as the pipeline, so results must match exactly.
3. pytest.raises checks that bad input really does raise the right error.
4. check() in the tour prints a result beside the pipeline's and marks it OK.

IF YOU BLANK

The tests prove the file matches Excel and the pipeline, that each algorithm agrees with a second method, and that bad data is refused.

WHERE: python_project.py, check line 935, tests in tests/test_python_project.py.



PYTHON PROJECT

python python_project.py

The full tour: each result beside the pipeline's, marked OK

python python_project.py --charts out/

Saves the Matplotlib and Seaborn charts

streamlit run python_project.py

Move A from 4 to 2: the share in P goes from 44% to 87%

python -m pytest tests/test_python_project.py

21 tests, in a few seconds

LIVE DEMO

IF A COMMAND FAILS

Show the saved output and charts in deliverables/assets/python, and keep going.

14

[Both · 1:30]

SAY

Abhay drives the terminal; Sankarshana explains. Everything runs on the frozen data, so the screen matches these slides.

WHAT THE CODE IS DOING

1. Tour: point at the OK marks against Excel and the pipeline in sections 3 and 4.

2. Charts: save them and open frontier.png.

3. Streamlit: move A from 4 to 2. The rupee split and VaR change (₹5,582 to ₹16,138). Each change reruns the script from the top. Then open Portfolio.complete in the editor and point at the y* line.

4. Pytest: run it and show all tests passing.

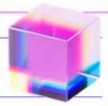
IF YOU BLANK

If a command fails, say so and move on to the saved output. Nothing else in the talk depends on it.

IF ASKED

If asked about any function, open it in the editor. docs/python_walkthrough.md has the function map and likely questions.

HONEST NOTES



WHAT CHANGED FROM THE APPROVED PROPOSAL, AND HOW WE USED AI, SO NOTHING IS HIDDEN.

Proposal said	The project does	Why
FINRA NCFS investor microdata	A 10-question questionnaire (capacity and willingness)	It sets A for one investor directly; we did not calibrate it on NCFS
Loughran–McDonald word lists	A small hedge-word set of the same kind	Keeps the file self-contained; it is not the full list
NSE total-return index	Nifty 50 price plus an estimated 1.22% a year	Estimated from the gap between Nifty BeES and the index
Anthropic SDK (optional)	A free model through OpenRouter (optional)	No cost for a live demo; every number still comes from our files

AI ASSISTANCE

Claude helped write and review this code. We checked it by running it, by testing it against Excel and the pipeline, and by reading it until we could explain each function; docs/python_walkthrough.md is our map. This is coursework, not investment advice.

[Abhay · 0:30]

SAY

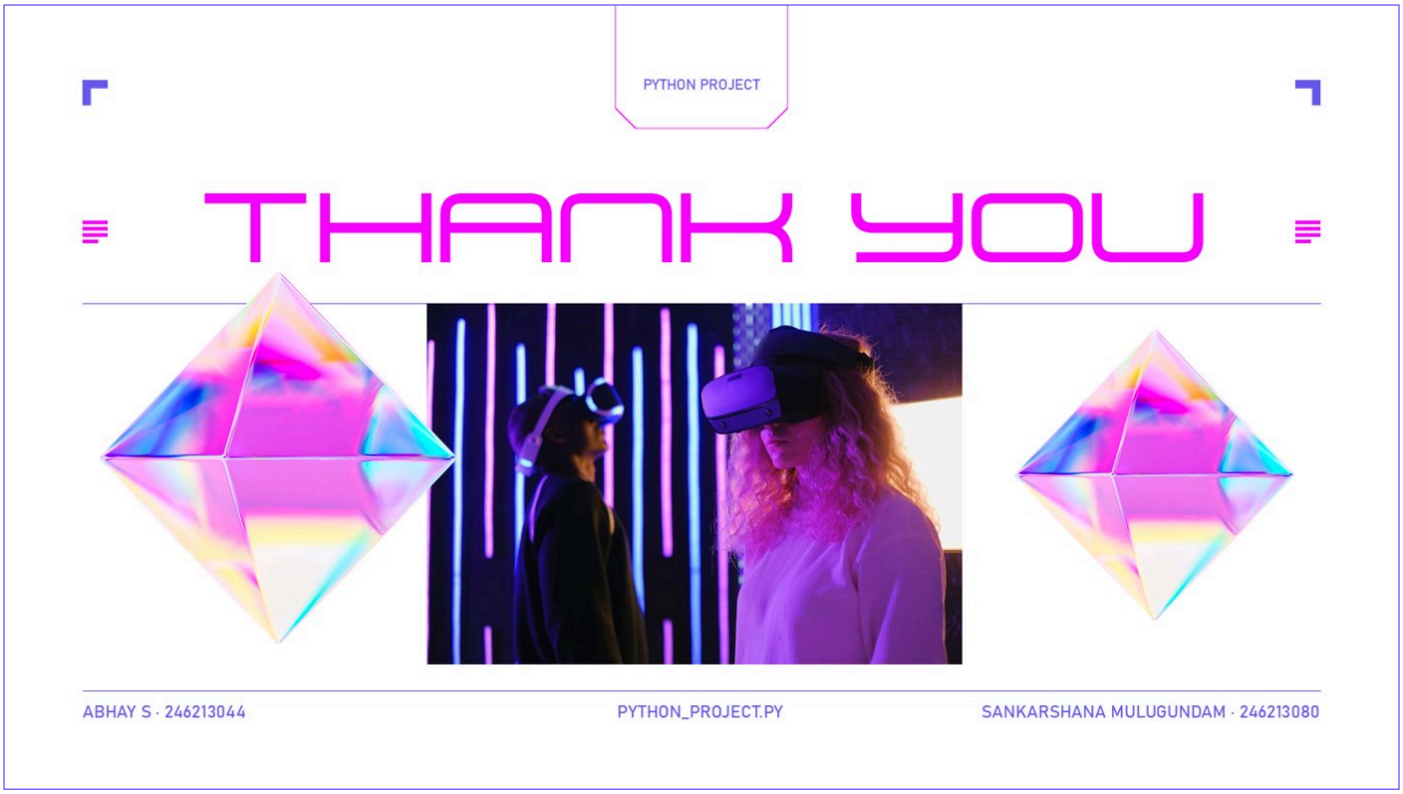
Two honest notes before we finish. First, four things differ from our proposal, and the table says what and why. Second, we used AI to help write and review this code. That is why we built so many independent checks, and why we made sure each of us can explain each function.

IF YOU BLANK

Here is what changed and why, and here is how we checked the code ourselves.

IF ASKED

Be honest about what you did yourself. The main results are checked by tests against Excel and the pipeline, not taken on an AI's word.



[Both · 0:10]

SAY

Thank you, and we'd welcome your questions.

IF YOU BLANK

Offer to open any function in the editor. It's the easiest way to answer a question about the code.